The Reasoning Gap in Automated Vulnerability Detection

A Comprehensive Benchmark of LLMs, SAST, DAST, and Graph-based Approaches

Medhat Hassan Ali[1], Youssef Sakr[2], Abdullah Eissa[3], Mohammed Aljazairy[4], Hisham Salah Rashad[5]

IEEE

IEEE Xplore® Digital Library

> whoami

Medhat Hassan (AKA m_1337)
Offensive Security Engineer & Researcher

- **Professional:** Cyber Security & Pen-Testing Program @ ITI
- **Academic:** Computer Engineering Graduate @ AAST
- **Focus Areas:** Web/API/Mobile Penetration Testing, Reverse Engineering, and AI-assisted Vulnerability Research.
- **Certifications:** eMAPT, eWPTX, CBBH, Practical Ethical Hacking.

The Core Challenges in Vulnerability Detection

86%

The Scale of the Problem

81% of the open-source codebases contain Critical vulnerabilities, creating a vast attack surface for the entire industry

63%

Failing Security Tools

Current tools generate overwhelming false positives, wasting countless hours of valuable developer time.

100x

Slow & Costly Remediation

Post-deployment bug fixes cost 100x more than finding them early, creating a slow and expensive remediation cycle.

73%

The Rise of AI Generated Code

AI-assisted coding is creating novel vulnerabilities faster than traditional security tools can adapt.

Our Comprehensive Evaluation Framework

provide a systematic approach to software analysis.
By integrating these complementary tools the framework ensures deeper insights

SAST Tools

CodeQL – SonarQube – Joern
Clang - Mend

Graph Models

Devign – Ample

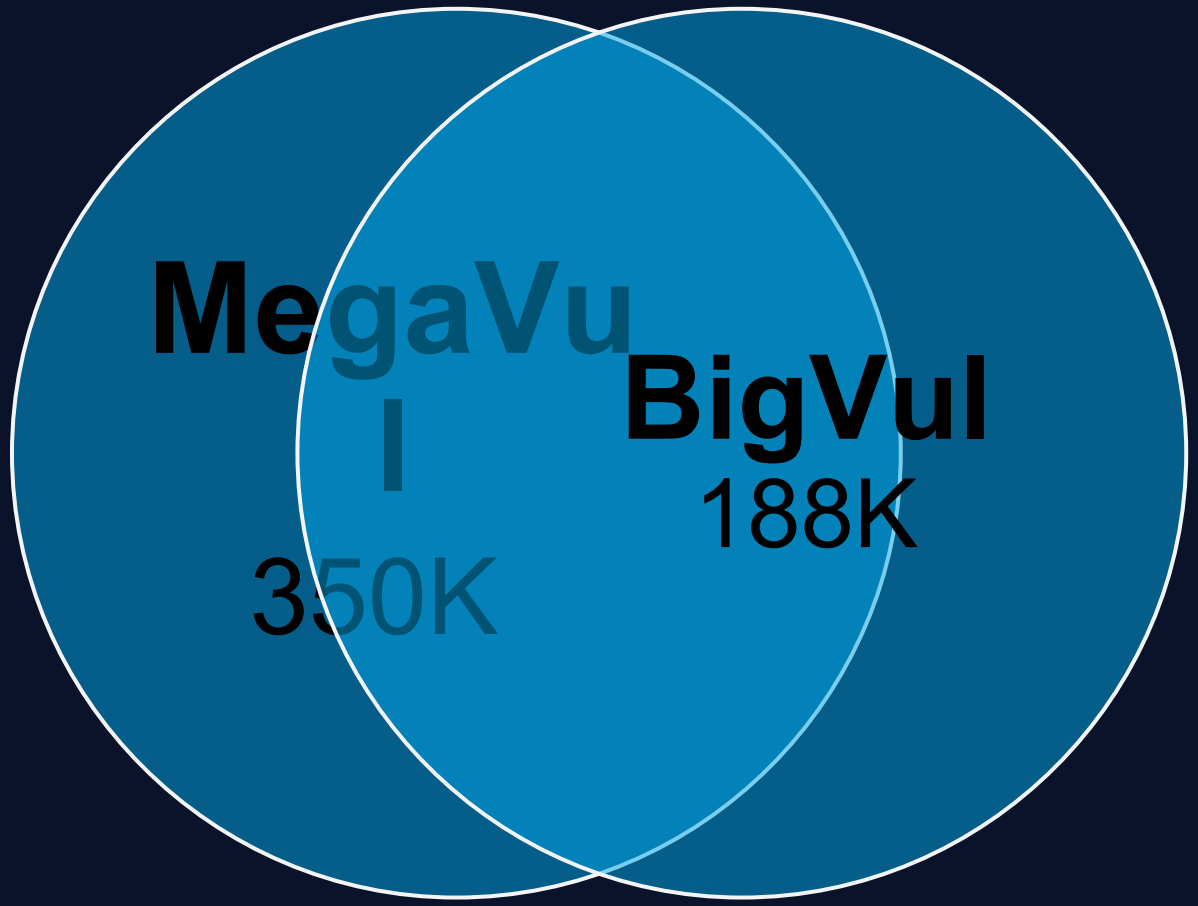
DAST Tools

Honggfuzz – MOpt-APL – AFL++

LLMs

GPT-4o mini – DeepSeek v3.2
Llama 3.2 - Gemma

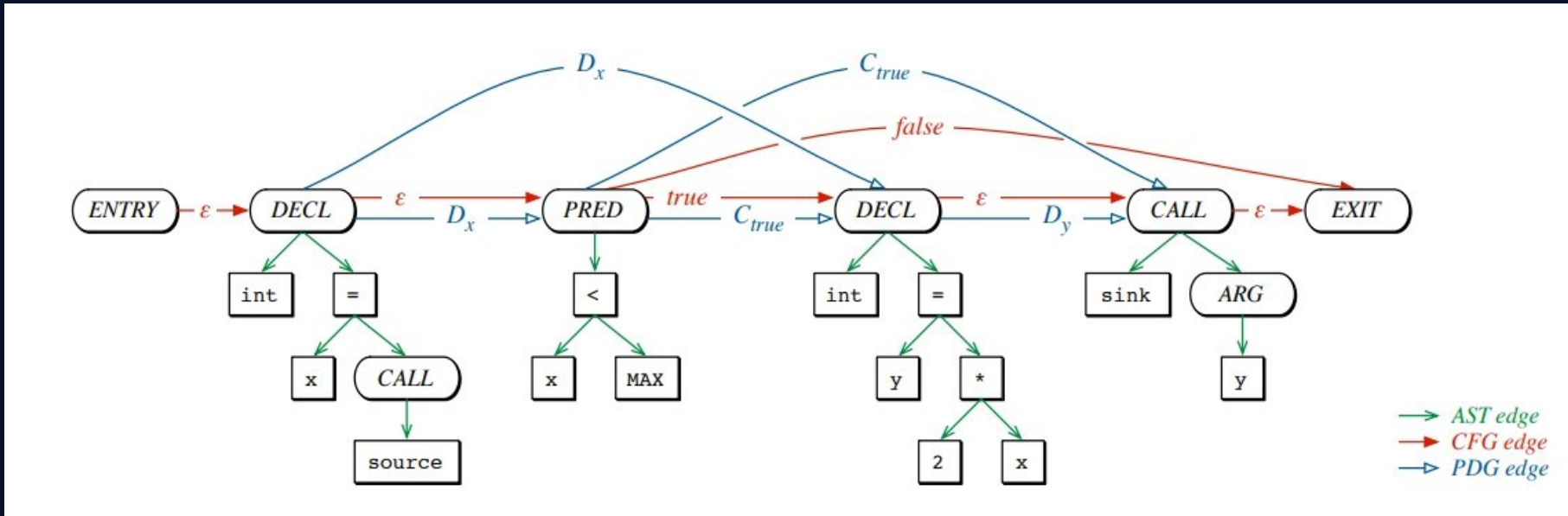
The Dataset Illusion: Benchmark Integrity



- AI models are only as good as their training data.
- **Overlap Discovered: 62%** of BigVul's unique hashes exist within the MegaVul dataset.
- **The Danger:** 95% of these intersected hashes are *vulnerable functions*.

Why Representation Matters: Text vs. Graphs

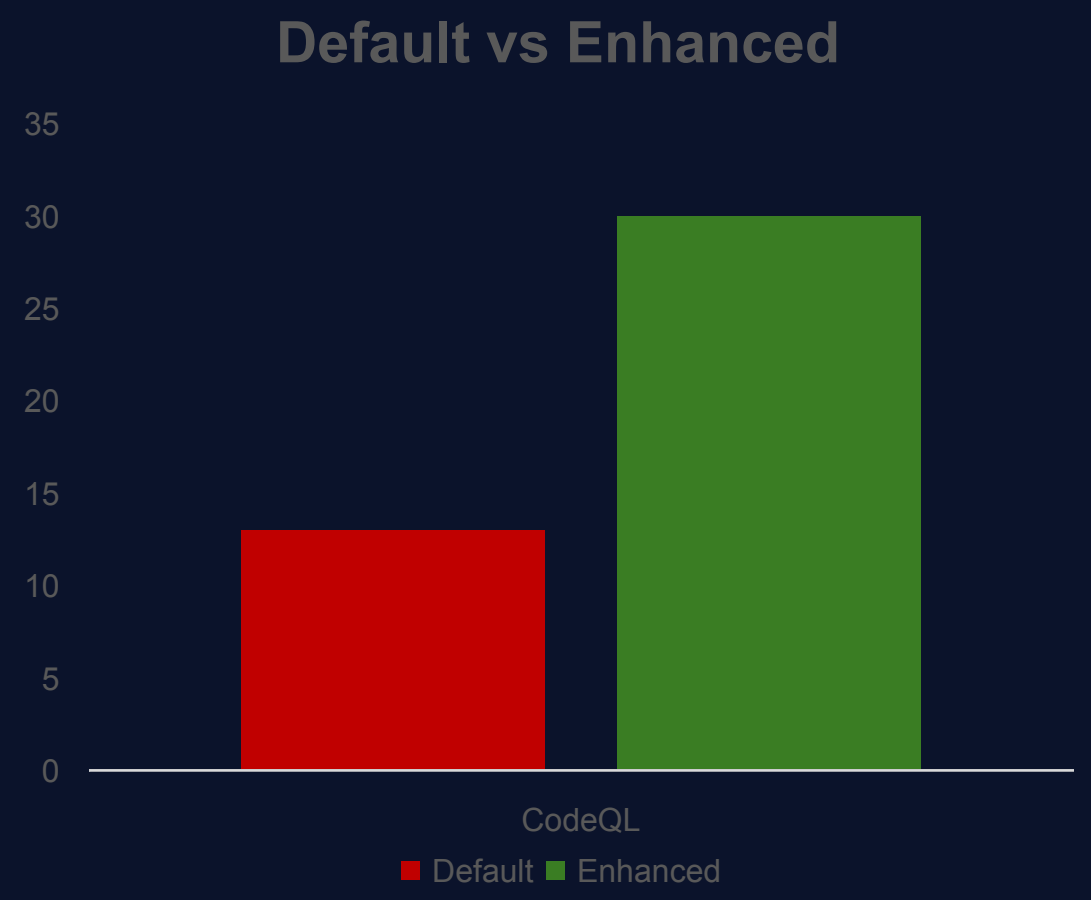
```
void foo()
{
    int x = source();
    if (x < MAX)
    {
        int y = 2 * x;
        sink(y);
    }
}
```



Graph-Based (CPG): Unifies Syntax (AST), Control Flow (CFG), and Data Dependencies (PDG) which enables robust taint analysis and tracks deep semantic context across multiple functions.

SAST & The Human Element

- Query-SAST > Non-Query Deep semantic understanding beats rigid pattern matching.
- Human-in-the-Loop: By manually tailoring queries.
- Result: A 130% Improvement in detection capability.



Dynamic Analysis (DAST) Realities

- **The Latency Gap: Reaching** a vulnerable code path is fast but ***triggering*** the fault takes hours.
- **The Reality Check:** Code coverage does **not** guarantee detection. Many bugs survived our 24-hour fuzzing campaigns.
- **The Cost:** Massive CPU/Memory overhead impacts scalability for large systems.

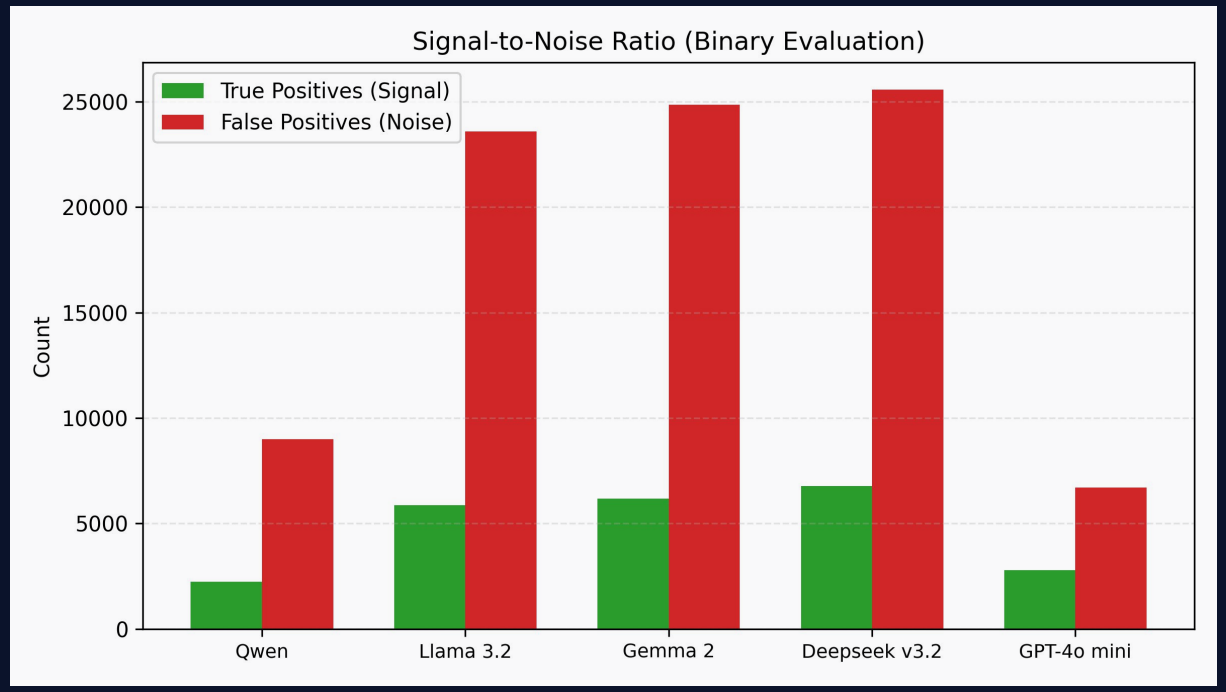
Reach (Seconds) → **Trigger (Hours / Never)**

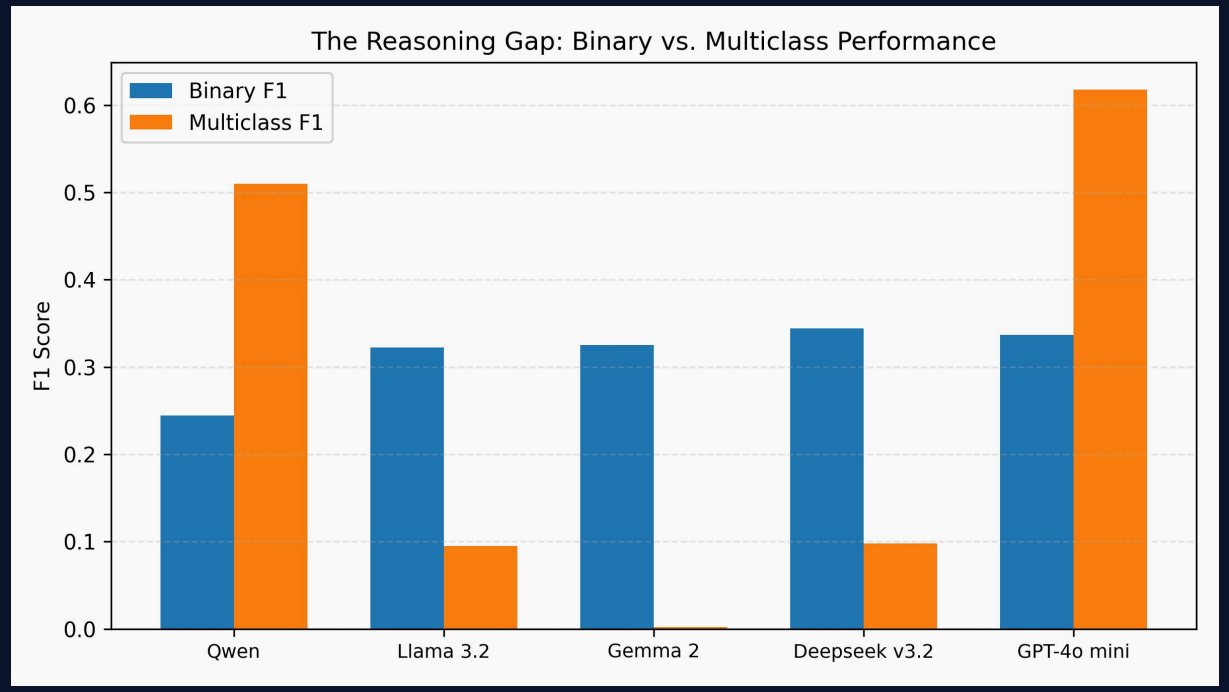
LLMs: The “Binary” Trap

Model	TP	TN	FP	FN	Prec	Rec	F1	Acc
Qwen	2230	19362	9010	4785	19.84%	31.79%	0.2443	61.02%
Llama 3.2	5876	4787	23585	1139	19.95%	83.76%	0.3222	30.13%
Gemma 2	6186	3509	24863	829	19.92%	88.81%	0.3250	27.40%
DeepSeek v3.2	6784	2788	25584	231	20.96%	96.71%	0.3445	27.05%
GPT4o mini	2780	21660	6712	4235	29.29%	39.63%	0.3368	69.06%

- Open-weight models (DeepSeek, Gemma 2) achieve massive **Recall (>88%)**
  They achieve this by predicting *everything* is vulnerable, resulting in a **False Positive Rate > 90%**.
- **GPT-4o mini** provided the most balanced profile (Highest F1: 33.68%, manageable FPR).

Multiclass & “The Reasoning Gap”





- **Performance Collapse:** Models with high binary recall saw their F1-scores plummet below **0.1**.
- **The Reasoning Gap:** Open-weight models lack the deductive reasoning to pinpoint *why* code is vulnerable.

The Deployment Roadmap

LLMS

Role: Rapid Auditing & Assistance
Action: Best used for pre-filtering code and assisting human developers

Q-SAST

Role: CI/CD Integration
Action: Deploy in large, evolving codebases. Requires manual query refinement by security engineers

DAST

Role: Deep Verification
Action: Reserve for applications handling complex memory allocations to generate actual PoV

Future Work & The Next Frontier

- **Hybrid Frameworks:** integrating AI-driven insights with structural analysis.
- **Scalability:** Adopting Quantization to LLMs and Graph simplification for CPG creation.
- **Dataset :** Developing a massive real-world dataset categorized by CWEs.

Next-generation frameworks that combine LLMs capabilities with the precision of graph-based representations to address the evolving complexity of modern exploits.

The future of automated security requires shifting from linear code reading to semantic reasoning.

Thank You

LinkedIn: in/medhat-hassan

GitHub: @medhathassn